

**UNIVERSIDADE VEIGA DE ALMEIDA**

Curso: Engenharia de Software

Disciplina: Paradigmas de Linguagens de Programação

**SISTEMA DE NOTAS E MÉDIAS DE ALUNOS**

Implementação nos paradigmas estruturado, orientado a objetos e funcional

José Victor Alves dos Santos - 1260123122

Gabriel da Silva Andrade - 1260131472

Gabriel Faustino de Oliveira - 1260135647

Tomaz Bontempo - 1210201858

Trabalho apresentado à disciplina de Paradigmas de Linguagens de Programação como requisito parcial de avaliação na Universidade Veiga de Almeida.

Professor: Rogerio Mandelli

Rio de Janeiro

2026

## **1 INTRODUÇÃO TEÓRICA**

### **1.1 Paradigmas de programação**

Um paradigma de programação é um conjunto de conceitos, princípios e práticas que definem a forma como um problema computacional é modelado e resolvido. Cada paradigma oferece uma perspectiva distinta sobre a organização do código, o fluxo de controle e a estrutura dos dados, influenciando diretamente a legibilidade, a manutenibilidade e a expressividade de uma solução. Este trabalho explora três paradigmas fundamentais: o estruturado, o orientado a objetos e o funcional, todos implementados na linguagem Python.

### **1.2 Paradigma estruturado**

O paradigma estruturado, consolidado na década de 1960 a partir dos trabalhos de Edsger W. Dijkstra e outros pesquisadores, organiza o programa em torno de funções e procedimentos independentes que operam sobre dados passados como argumentos. O controle do fluxo é realizado exclusivamente por meio de três estruturas: sequência, seleção e repetição, eliminando o uso do comando goto e tornando o código mais previsível e legível.

O princípio central do paradigma é a modularização: cada função realiza uma tarefa específica e bem delimitada, podendo ser reutilizada em diferentes partes do programa. A separação clara entre entrada de dados, processamento e saída de resultados facilita a compreensão, o teste e a manutenção do código. Em Python, esse paradigma é implementado por meio de funções definidas com `def`, sem o uso de classes ou objetos.

### **1.3 Paradigma orientado a objetos**

O paradigma orientado a objetos (POO) organiza o programa em torno de objetos, que são instâncias de classes. Uma classe define um molde contendo atributos (dados) e métodos (comportamentos), permitindo que cada objeto encapsule seu próprio estado interno e exponha apenas as operações necessárias ao mundo externo. Esse modelo busca representar entidades do mundo real de forma mais natural e intuitiva dentro do código.

Os quatro pilares fundamentais do paradigma são o encapsulamento, que protege os dados internos de acesso direto; a herança, que permite que uma classe filha reutilize e especialize o comportamento de uma classe mãe; o polimorfismo, que permite que diferentes classes respondam de forma distinta ao mesmo método; e a abstração, que oculta detalhes de implementação, expondo apenas o necessário. Em Python, esses conceitos são implementados por meio de classes, métodos especiais como `__init__` e `__str__`, properties com `@property` e a instrução `super()`.

#### **1.4 Paradigma funcional**

O paradigma funcional trata a computação como a avaliação de funções matemáticas e evita a alteração de estado e dados mutáveis. Seu princípio central é o uso de funções puras, que para um mesmo conjunto de entradas sempre produzem a mesma saída e não causam efeitos colaterais. Isso torna o código mais previsível, testável e paralelizável.

Os principais recursos funcionais utilizados neste trabalho são as funções lambda, que permitem definir funções anônimas em uma única linha; map, que aplica uma função a cada elemento de uma lista; filter, que seleciona elementos com base em uma condição; reduce, que acumula os elementos de uma lista em um único valor; e recursão, que substitui estruturas de repetição por chamadas sucessivas à própria função. Em Python, esses recursos estão disponíveis nativamente, com reduce importado do módulo `functools`.

#### **1.5 A linguagem Python**

Python é uma linguagem de programação multiparadigma de alto nível, interpretada e de tipagem dinâmica, criada por Guido van Rossum e lançada em 1991. Sua sintaxe clara e expressiva permite que o mesmo problema seja resolvido de maneiras distintas dentro de um único ambiente, tornando-a ideal para a comparação entre paradigmas. Python suporta nativamente os paradigmas estruturado, orientado a objetos e funcional, o que justifica sua escolha como linguagem única para as três implementações deste trabalho.

## **2 DESCRIÇÃO DO PROBLEMA**

O problema escolhido pelo grupo é o Problema 1: Sistema de Notas e Médias de Alunos. O sistema foi desenvolvido para demonstrar as diferenças conceituais entre os três paradigmas ao resolver um problema prático e familiar, com requisitos claros e bem delimitados.

O sistema deve ser capaz de ler o nome e as notas de um número variável de alunos informado pelo usuário; calcular a média aritmética individual de cada aluno a partir de suas notas; determinar a situação de cada aluno como Aprovado, quando a média for igual ou superior a 6,0, ou Reprovado, quando for inferior; exibir a listagem completa dos alunos ordenada de forma decrescente por média; e calcular e exibir estatísticas gerais da turma, incluindo a média geral, o aluno com a maior média e o aluno com a menor média.

O mesmo conjunto de requisitos foi implementado três vezes, cada uma utilizando um paradigma distinto, para evidenciar como cada abordagem modela e resolve o mesmo problema de forma diferente. O programa principal, contido no arquivo `main.py`, exibe um menu interativo que permite ao usuário escolher o paradigma desejado e executar a solução correspondente, retornando ao menu ao final de cada execução.

### **3 IMPLEMENTAÇÃO COMENTADA**

#### **3.1 Estrutura geral do projeto**

O projeto é organizado em quatro módulos separados. O arquivo `main.py` contém o menu principal e utiliza a instrução `match/case` do Python 3.10 para redirecionar a execução ao módulo correspondente ao paradigma escolhido. Os módulos `estruturado/notas.py`, `orientado_objetos/` e `funcional/notas.py` contêm, respectivamente, as implementações dos três paradigmas. Cada módulo possui sua própria função `main()` com um loop interno que pergunta ao usuário se deseja executar novamente antes de retornar ao menu principal.

#### **3.2 Paradigma estruturado**

A implementação estruturada é organizada em cinco funções independentes, cada uma responsável por uma etapa específica do processo. A função `ler_alunos()` realiza a entrada de dados, coletando nome e notas e retornando uma lista de

dicionários. A função `calcular_media()` recebe uma lista de notas e retorna a média aritmética. A função `determinar_situacao()` recebe a média e retorna a string correspondente à situação do aluno. A função `exibir_lista_ordenada()` ordena a lista de alunos por média de forma decrescente e exibe os resultados. Por fim, a função `calcular_estatisticas()` percorre a lista para calcular e exibir a média geral da turma, o aluno com a maior e o de menor média.

A principal decisão de implementação neste paradigma foi separar explicitamente as responsabilidades de entrada, processamento e saída em funções distintas. O uso de dicionários Python para representar cada aluno substitui o uso de objetos, mantendo a solução dentro dos limites do paradigma. A ordenação é realizada com `sorted()` e uma função lambda como chave, recurso disponível em Python mesmo em contexto estruturado.

### 3.3 Paradigma orientado a objetos

A implementação orientada a objetos é distribuída em três arquivos. A classe `Pessoa`, definida em `pessoa.py`, funciona como classe base e implementa o encapsulamento do atributo `nome` por meio de um atributo privado com duplo underscore e um par de métodos `@property` e `@setter` que validam o valor antes de atribuí-lo. O método `__str__` define a representação textual padrão de qualquer instância da classe.

A classe `Aluno`, em `aluno.py`, herda de `Pessoa` por meio da instrução `class Aluno(Pessoa)` e chama o construtor da classe mãe com `super().__init__(nome)`. Ela adiciona o atributo privado `__notas` com validação de tipo por meio de um setter, o método `calcular_media()` que usa `try/except` para tratar a divisão por zero em caso de lista vazia, e o método `situacao()`. O método `__str__` é sobrescrito para exibir as informações completas do aluno, demonstrando o polimorfismo: o mesmo nome de método produz saídas diferentes dependendo da classe do objeto.

A classe `Turma` agrega objetos do tipo `Aluno` em uma lista privada e oferece os métodos `adicionar_aluno()`, que valida o tipo do argumento com `isinstance()` antes de aceitar o objeto, `media_geral()`, `maior_nota()`, `menor_nota()` e `listar_ordenado()`. O arquivo `orientado_objetos/main.py` instancia um objeto `Turma`, coleta os dados do usuário, cria objetos `Aluno` e delega todo o processamento aos métodos da classe.

### 3.4 Paradigma funcional

A implementação funcional evita qualquer alteração de estado externo e não utiliza classes ou objetos. Todas as operações são realizadas por meio de funções puras e dos recursos funcionais do Python. A função `media_recurativa()` calcula a média sem utilizar estruturas de repetição, acumulando a soma das notas por meio de chamadas recursivas até atingir o caso base, quando o índice atinge o comprimento da lista.

As funções `lambda` aprovado e `situacao` encapsulam as regras de negócio de aprovação em expressões de uma linha. A função `processar_alunos()` usa `map` para transformar a lista de dicionários de entrada em uma nova lista com média e situação calculadas para cada aluno, sem modificar a lista original. As funções `filtrar_aprovados()` e `filtrar_reprovados()` usam `filter` para selecionar subconjuntos da lista processada. As funções `media_geral()`, `maior_nota()` e `menor_nota()` usam `reduce` para acumular a soma das médias e comparar pares de alunos, respectivamente. A função `ordenar_por_media()` usa `sorted()` com uma `lambda` como chave, retornando uma nova lista ordenada sem alterar a original, respeitando o princípio de imutabilidade dos dados.

## 4 ANÁLISE COMPARATIVA

### 4.1 Tabela comparativa

| <b>Critério</b>      | <b>Estruturado</b> | <b>Orientado a Objetos</b> | <b>Funcional</b>  |
|----------------------|--------------------|----------------------------|-------------------|
| Legibilidade         | Alta               | Média/Alta                 | Média             |
| Modularização        | Por funções        | Por classes                | Por funções puras |
| Reutilização         | Média              | Alta (herança)             | Alta (composição) |
| Manutenção           | Simples            | Média                      | Simples           |
| Curva de aprendizado | Baixa              | Alta                       | Média/Alta        |
| Testabilidade        | Média              | Média                      | Alta              |
| Volume de código     | Baixo              | Alto                       | Médio             |

## 4.2 Legibilidade e produtividade

A implementação estruturada demonstrou a maior legibilidade para um problema desta natureza. A sequência linear de funções com nomes descritivos torna o fluxo do programa imediatamente compreensível, mesmo para leitores sem experiência com o paradigma. A ausência de abstrações adicionais, como classes e decoradores, reduz o esforço cognitivo necessário para entender o código.

A implementação orientada a objetos exige maior esforço inicial de projeto, pois é necessário definir a hierarquia de classes e as responsabilidades de cada uma antes de escrever qualquer lógica de negócio. Contudo, uma vez que a estrutura está estabelecida, a adição de novos comportamentos torna-se mais natural e organizada, especialmente em projetos de maior escala.

A implementação funcional apresenta a curva de aprendizado mais elevada entre as três, pois requer familiaridade com conceitos como funções de ordem superior, imutabilidade e recursão. O código resultante é conciso, mas pode ser menos intuitivo para quem não está habituado ao paradigma.

## 4.3 Tipos de dados, escopo e persistência

As três implementações utilizam os mesmos tipos de dados nativos do Python: listas, dicionários, strings e números de ponto flutuante. A diferença está na forma como esses dados são organizados e acessados. No paradigma estruturado, os dados são representados por dicionários simples e passados diretamente entre as funções como argumentos. No paradigma orientado a objetos, os dados são encapsulados dentro dos objetos e acessados exclusivamente por meio de métodos e properties, protegendo a integridade interna do estado. No paradigma funcional, os dados de entrada nunca são modificados: cada função recebe uma coleção e retorna uma nova coleção transformada, respeitando o princípio de imutabilidade.

Em relação ao escopo, todas as variáveis são locais às suas respectivas funções ou métodos nas três implementações, sem uso de variáveis globais. A persistência de dados está limitada à execução do programa em todas as versões, sem uso de arquivos ou bancos de dados, conforme o escopo do problema proposto.

## 4.4 Diferenças no comportamento das linguagens

Por serem todas implementadas em Python, as três versões compartilham o mesmo comportamento de execução, tipagem dinâmica e gerenciamento de memória por coleta de lixo. A diferença mais relevante no comportamento da linguagem entre os paradigmas está no uso de recursão no paradigma funcional: Python possui um limite padrão de profundidade de recursão de 1.000 chamadas, o que tornaria a função `media_recursiva()` inadequada para turmas muito grandes. Em contextos de produção, esse limite poderia ser contornado com `sys.setrecursionlimit()` ou com uma abordagem iterativa com acumulador.

Outro comportamento relevante é o uso de `sorted()` nas três implementações. A função retorna sempre uma nova lista, sem modificar a original, comportamento que é especialmente valorizado no paradigma funcional, mas presente igualmente nas outras versões por ser uma característica da própria linguagem.

## **5 DISCUSSÃO SOBRE CONCORRÊNCIA**

Embora a implementação atual seja inteiramente sequencial e voltada para uso por um único usuário por vez, é possível discutir evoluções futuras da solução que envolvam concorrência e múltiplos usuários.

Uma evolução natural seria transformar o sistema em uma aplicação web ou de rede capaz de receber dados de múltiplos usuários simultaneamente. Nesse cenário, cada requisição de processamento poderia ser tratada como uma tarefa independente. O paradigma funcional se mostraria especialmente adequado para essa evolução, pois a ausência de estado compartilhado e o uso de funções puras eliminam as condições de corrida (race conditions) que surgem quando múltiplas threads acessam e modificam os mesmos dados simultaneamente.

No paradigma orientado a objetos, a introdução de concorrência exigiria cuidado especial com o acesso concorrente aos atributos dos objetos, sendo necessário o uso de mecanismos de sincronização como locks ou o uso de estruturas de dados thread-safe. No paradigma estruturado, a ausência de estado encapsulado facilitaria a paralelização, desde que os dados de cada usuário fossem mantidos isolados em seus respectivos escopos de execução.

Em Python, a concorrência pode ser implementada por meio dos módulos `threading`, `multiprocessing` ou `asyncio`, sendo este último especialmente adequado



para aplicações de entrada e saída intensiva, como um servidor que recebe notas de múltiplos usuários por meio de uma interface web.

## 6 CONCLUSÃO

A implementação do Sistema de Notas e Médias de Alunos nos três paradigmas permitiu observar, na prática, as diferenças conceituais e estruturais entre as abordagens estudadas ao longo da disciplina. Cada paradigma revelou vantagens e limitações distintas diante do mesmo problema.

O paradigma estruturado mostrou-se o mais adequado para este problema específico, por se tratar de um sistema de escopo limitado, com fluxo linear e sem necessidade de reutilização extensiva de código. A simplicidade das funções independentes, a clareza do fluxo de dados e a ausência de abstrações desnecessárias tornam a solução estruturada a mais legível e direta entre as três.

O paradigma orientado a objetos demonstrou maior potencial de escalabilidade. A hierarquia de classes Pessoa e Aluno, o encapsulamento dos dados e o polimorfismo de `__str__` constroem uma base sólida sobre a qual novos tipos de usuários, como professores ou coordenadores, poderiam ser adicionados com esforço mínimo. Para sistemas maiores e mais complexos, o paradigma orientado a objetos seria a escolha mais adequada.

O paradigma funcional evidenciou a elegância do pensamento declarativo: em vez de descrever como o programa deve executar cada passo, as funções descrevem o que deve ser obtido a partir dos dados. O uso de `map`, `filter` e `reduce` tornou o código conciso e altamente testável, e a imutabilidade dos dados seria um diferencial importante em cenários de concorrência. A principal limitação observada foi a curva de aprendizado mais elevada e a restrição do limite de recursão do Python para entradas muito grandes.

Como principal aprendizado da disciplina, este trabalho reforça que a escolha do paradigma não deve ser dogmática, mas guiada pelo contexto do problema: sua escala, sua complexidade, o perfil da equipe e os requisitos de manutenção e evolução futura do sistema.